The Journal of Open Source Software

LinkingLines: Using the Hough Transform to Cluster Line Segments and for Mesoscale Feature Extraction

Allison Kubo Hutchison [1], **Leif Karlstrom** [1], **and Tushar Mittal** [2]

1 University of Oregon **1** Department of Earth Sciences, University of Oregon, Eugene, OR, USA **2** Department of Geosciences, Pennsylvania State University, University Park, PA, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Hugo Ledoux

Reviewers:

- @evetion
- @nialov

Submitted: 30 October 2023
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Linear feature analysis plays a fundamental role in various scientific and geospatial applications, from detecting infrastructure networks to characterizing geological formations. In this paper, we introduce `linkinglines`, an open-source Python package tailored for the clustering, and feature extraction of linear structures in geospatial data. Our package leverages the Hough Transform, commonly used in image processing, performs clustering of line segments in the Hough Space then provides unique feature extraction methods and visualization. `linkinglines` empowers researchers, data scientists, and analysts across diverse domains to efficiently process, understand, and extract valuable insights from linear features, contributing to more informed decision-making and enhanced data-driven exploration. We have used `linkinglines` to map dike swarms with thousands of segments associated with Large Igneous Provinces in Kubo Hutchison et al. (2023). T

Statement of Need

The `linkinglines` Python package addresses the need for quantitative and automated line clustering and analysis in geologic data. As the volume of data continues to grow, including remote sensing, computer vision, and geographic information systems (GIS), there is an increasing demand for tools that can simplify the extraction and analysis of linear features such as dikes, fractures, roads, lineaments.

The primary needs that the `linkinglines` package fulfills include:

1. **Dissected Line Extraction or Data Reduction**: In areas where land cover or data availability effects the complete mapping of linear features, this package can link together similarly oriented segments into lines in a quantitative automated way. This is also an effective data reduction technique.

2. **Feature Extraction and Analysis**: The `linkinglines` package provides functions to extract and then compute essential metrics and statistics on extracted linear, radial or circumferential type features which are commonly seen in dike swarms or fracture networks.

3. **Geospatial and Image Data Integration**: Geospatial data often involves complex relationships between various linear features. `linkinglines` integrates seamlessly with popular geospatial libraries like `pyproj` and `geopandas` to facilitate georeferenced data analysis and visualization.

4. **Custom Plotting and Visualization**: Effective visualization is critical for data interpretation. The package offers custom plotting scripts, making it easier to visualize results and communicate findings effectively.

Statement of the Field

This package was originally developed to tackle the issue of mapped dike segments. Rugged terrain, vegetation cover, and a large area made it impossible to accurately map dikes. The length, density, and structure of the dike swarm affects how magma is transported and erupted. Scaling analysis indicated that for segments of widths of 10 m, dikes could be 10-10s of kilometers long however observed segments were two orders of magnitude lower (Morriss et al., 2020). Additionally, the complex overlapping structure of the dike swarm was difficult to analyze. We designed `linkinglines` to extract not only lines from line segments but also help analyze the mesoscale structure of the dike swarm. Using the unique properties of the Hough Transform we can extract several unique mesoscale structures within a group of lines.

Issues with data coverage, incompleteness, and complexity occur in a wide range of Earth and Planetary science data products. As an example we show how this work can also be applied to lineaments on Venus(**venus?**) and fracture networks(Ovaskainen et al., 2023) in our documentation. Clustering and machine learning methods are commonly applied geosciences within the subdisciplines of seismology, geochemistry, and planetary science (**li20f23machine?**). There is currently available code to work with geospatial data clustering such as `geopandas`, `PySAL`, and others which perform the Hough Transform on images such as `scikit-image` or `OpenCV`, however this seems to be the first union of the two methods and specific to geoscience data applications (Bradski, 2000; Gaboardi et al., 2021; Jordahl et al., 2020; Walt et al., 2014). There is a clear use case for `linkinglines` to be applied to planetary science and the variety of linear features on Venus and Mars. For example, `linkinglines` can be used to quantitatively identify the centers of radial lineaments on Venus and relate the data back to volcanism and plate tectonics, although work has been done to map and qualitatively characterize these features `linkinglines` offers unique insight and quantitative measure (**erst?**). Related packages which are specific to geosciences such as `fractopo` which focuses on network analysis of fractures could be utilized in parallel with `linkinglines` for additional analysis (Ovaskainen, 2023).

Algorithm

`linkinglines` can read in any common geospatial data format using `geopandas` such as ESRI Shapefiles, GeoJSON, or Well Known Text (Jordahl et al., 2020). Preprocessing is applied to the dataset so that only straight lines are considered in the algorithm. This is done by loading in the points from each vector object and performing linear regression, only considering those that yield a line with a $p > 0.05$. The data is then formatted into a `pandas` DataFrame. This package heavily uses `pandas` as the database structure for ease of use, data manipulation, and integration with `numpy` and `scipy`(McKinney, 2010).

DRAFT

The Journal of Open Source Software

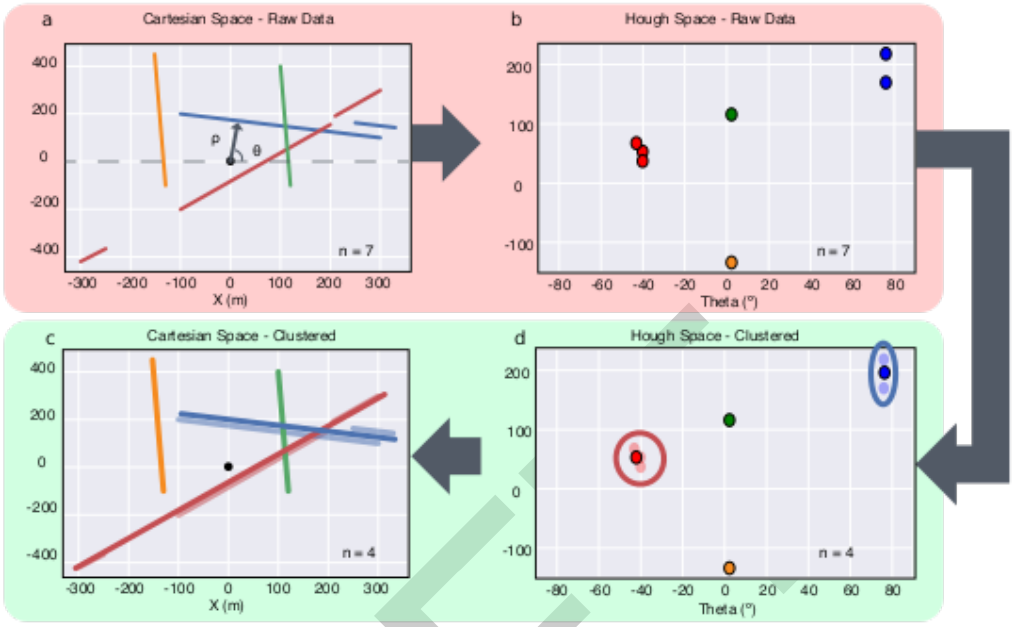

Figure 1: Dike linking algorithm using the Hough Transform. First, raw data in Cartesian space is converted into Hough space (a and b). Agglomerative clustering is then performed on the data in Hough coordinates (d). In this example, there are four dikes total and two (red and blue) clusters. The clusters are redrawn by connecting the endpoints of the segments in the cluster (c).

The Hough Transform is an image processing technique used for detecting straight lines and other patterns in binary or edge-detection images(Duda & Hart, 1972). It achieves this by converting points in an image into parametric equations and identifying patterns through the accumulation of votes in a parameter space. The transform has been generalized to detect arbitrary shapes, making it a versatile tool for pattern recognition and image analysis (Ballard, 1981). Applications of the Hough Transform include object or motion detection, lane tracking for cars, road detection in geospatial data, and biometric authentication (Mukhopadhyay & Chaudhuri, 2015). After loading in the data, it is assumed to be already line structures, so the accumulator array of the Hough Transform is skipped, although this functionality could be added if needed. First, the angle of the line segment is found. In many methods, it is the left-hand corner of the image (Ballard, 1981), but we choose the average midpoint of the line segments (Figure 1B). Other origins can be specified in certain functions using the `xc` and `yc` arguments. We use the ρ, θ parameterization of (Duda & Hart, 1972).

After the coordinate transform, ρ and θ become the basis for the Agglomerative clustering step, where we utilize Scipy's clustering algorithm (Virtanen et al., 2020). The clustering algorithm takes two inputs: `dtheta` and `drho`, which are used to scale the Hough Transform data. Then the clustering distance is set to 1. Combined with the default complete linkage scheme, effectively, this prevents clusters from being formed that have ranges greater than either `dtheta` or `drho` and the linear combination of the two where:

$$d = \sqrt{\left(\frac{\theta_1 - \theta_2}{d\theta}\right)^2 + \left(\frac{\rho_1 - \rho_2}{d\rho}\right)^2} \tag{3}$$

where two members of a potential cluster are denoted by the subscripts 1 and 2. Other linkage or distance schemes could be considered and implemented based on the specific research applications.

DRAFT

After labels are assigned in the clustering portion of the algorithm, new lines are drawn using the endpoints of the clustered lines (Figure 1D). For each cluster, the nearest neighbors of the segment midpoints are calculated in Cartesian space, allowing for an analysis of the Cartesian spatial clustering of the lines. We also introduce an optional filtering step, which analyzes the maximum nearest neighbor difference of midpoints normalized by the total cluster length. This filters out clusters with segments that are not evenly clustered in Cartesian space.

Feature Extraction

Additionally, we leverage the unique properties of the Hough Transform to combine clustering with feature extraction. In the original usage case of overlapping complex dike swarms, two potential end members of swarm types are linear and radial or circumferential swarms (Figure 2). We can easily derive equations to describe these Cartesian patterns in the Hough Space, then perform a best-fit analysis using Virtanen et al. (2020).

DRAFT

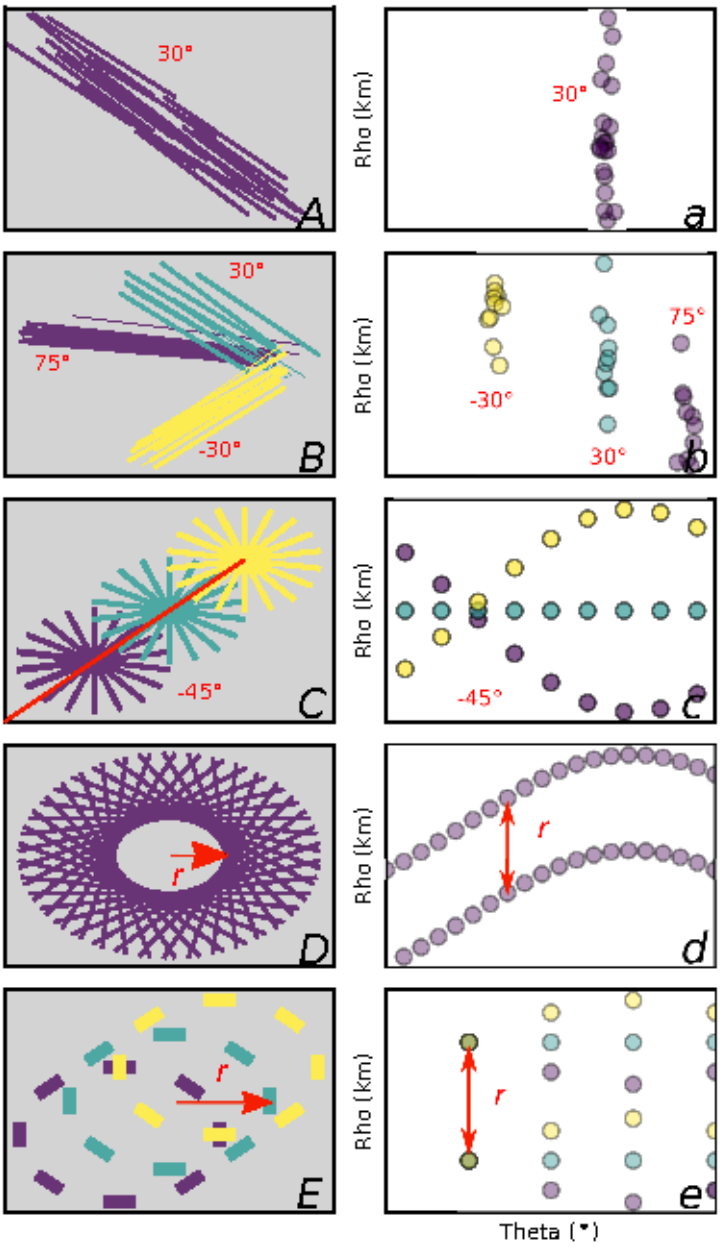

Figure 2: Synthetic dike swarms in a Cartesian space (gray background, uppercase label) and Hough Transform space (white background, lowercase label). (A) Shows a simple linear swarm oriented at 30°. (B) Shows three linear swarms at −30°, 30°, 75°. (C) Shows three radial swarms aligned at a −45° angle. The angle at which radial swarms intersect in the Hough space (HS) is the angle of their relative orientation in Cartesian space. (D) Shows a circumferential swarm with the lines extending to show how it converges to Equation 8. The radius of the circumferential swarm is equal to the spacing of the parallel two curves in HS. (E) Shows three circumferential swarms with the same radius aligned at a -45° angle.

In the case of a radial or circumferential pattern, the equation is actually the Hough Transform equation where the radial values of ρ_r can be expressed as:

$$\rho_r(\theta) = (x_r - x_c)\cos(\theta) + (x_r - y_c)\sin(\theta) \tag{4}$$

where the radial form is a function of θ and the center of the radial form, a Cartesian location (x_r, y_r) (Figure 2). Armed with this equation, we can apply a best-fit analysis of the data to find quantitative center locations for radial or circumferential patterns (Figure 2D,E). In the application of dike data, this may point to a central magma chamber or locus of stress. We can also then remove the lines which fit those patterns for feature extraction.

For extraction of linear features there are two options, one is the clustering step described above, the second can be applied with looking for mesoscale (mid-scale) clusters, i.e. cluster of clusters. We apply the Hough accumulator array, a 2D histogram of θ and ρ. You can set the size of bins in the histogram and if clusters fall within those boxes they can be thought of as mesoscale clusters. We allow for flexibility of cutoffs for these mesoscale feature extraction, so it can be tailored to each research or engineering application.

Overall, these capabilities can be separated from the clustering and linking steps but is combined for ease of use in the `linkinglines` package.

Future Work

This package takes geospatial or other types of line segment data and clusters them based on their orientation. Currently, the Hough transform assumes that the data input into are only straight line segments however it could be generalized to arbitrary shapes for more flexibility (Ballard, 1981). Additionally, future work could incorporate other shapes or patterns in the Hough Space and could extend the feature extraction methods laid out here. We invite collaboration to increase the capabilities of this code.

Acknowledgements

This paper was made possible by the "Crafting Quality Research Software and Navigating Publication in Software Journals" held by the Computational Infrastructure for Geodynamics in September 2023.

References

Ballard, D. H. (1981). Generalizing the hough transform to detect arbitrary shapes. *Pattern Recognition*, *13*(2), 111–122. https://doi.org/10.1016/0031-3203(81)90009-1

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

Duda, R. O., & Hart, P. E. (1972). Use of the hough transformation to detect lines and curves in pictures. *Communications of the ACM*, *15*(1), 11–15. https://doi.org/10.1145/361237.361242

Gaboardi, J. D., Rey, S., & Lumnitz, S. (2021). Spaghetti: Spatial network analysis in PySAL. *Journal of Open Source Software*, *6*(62), 2826. https://doi.org/10.21105/joss.02826

Jordahl, K., Bossche, J. V. den, Fleischmann, M., Wasserman, J., McBride, J., Gerard, J., Tratner, J., Perry, M., Badaracco, A. G., Farmer, C., Hjelle, G. A., Snow, A. D., Cochran, M., Gillies, S., Culbertson, L., Bartos, M., Eubank, N., maxalbert, Bilogur, A., … Leblanc, F. (2020). *Geopandas/geopandas: v0.8.1* (Version v0.8.1). Zenodo. https://doi.org/10.5281/zenodo.3946761

Kubo Hutchison, A., Karlstrom, L., & Mittal, T. (2023). Multiscale spatial patterns in giant dike swarms identified through objective feature extraction. *Geochemistry, Geophysics, Geosystems*, *24*(9), e2022GC010842. https://doi.org/10.1029/2022GC010842

DRAFT

McKinney, W. (2010). Data structures for statistical computing in python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th python in science conference: SciPy 2010* (pp. 56–61). SciPy. https://doi.org/10.25080/Majora-92bf1922-00a

Morriss, M. C., Karlstrom, L., Nasholds, M. W., & Wolff, J. A. (2020). The chief joseph dike swarm of the columbia river flood basalts, and the legacy data set of william h. taubeneck. *Geosphere*, *16*(4), 1082–1106. https://doi.org/https://doi.org/10.1130/GES02173.1

Mukhopadhyay, P., & Chaudhuri, B. B. (2015). A survey of Hough Transform. *Pattern Recognition*, *48*(3), 993–1010. https://doi.org/10.1016/j.patcog.2014.08.027

Ovaskainen, N. (2023). Fractopo: A python package for fracture network analysis. *Journal of Open Source Software*, *8*(85), 5300. https://doi.org/10.21105/joss.05300

Ovaskainen, N., Skyttä, P., Nordbäck, N., & Engström, J. (2023). Detailed investigation of multi-scale fracture networks in glacially abraded crystalline bedrock at Åland islands, finland. *Solid Earth*, *14*(6), 603–624. https://doi.org/10.5194/se-14-603-2023

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Walt, S. van der, Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & contributors, the scikit-image. (2014). Scikit-image: Image processing in python. *PeerJ*, *2*, e453. https://doi.org/10.7717/peerj.453

DRAFT